Robo Views Bot

Full System Analytic

Organic-style Telegram post views drip via JustAnotherPanel
Generated: 8/8/2026, 10:00:06 PM
Local stack: Node.js · Express · JAP API · JSON store · Dashboard :3847

1. What is this system?

Robo Views Bot is a local Node.js drip engine that buys Telegram post views from JustAnotherPanel (JAP) on a timed schedule so growth looks organic (~1,200 views over ~4.5 hours). It includes a web dashboard and optional channel auto-discover.

It is NOT a Telegram bot and does not use the Bot API for views. It places SMM panel view orders. Channel scrape is only for discovery; the core product is the drip scheduler + JAP.

2. Architecture

High-level components and data flow:

```
Dashboard UI (public/*)  <% %  REST :3847 % % >  Express server (index.js)
                                                        |
          +-------------------------+-----------------+------------------+
          v                         v                                    v
   worker.js (every 2s)     logic/curve.js                     telegram.js
   drip timer / fire        schedule builder                   scrape t.me
          |
          v
   jap.js  % % >  JAP API service 7394  (+20 views)
          |
          v
   db.js  !'  data/db.json  (posts · drops · orders · events · settings)
```

Piece	
server/logic/curve.js	Builds full timed schedule of +20 drops
server/worker.js	Every 2s: due drops !' place orders
server/jap.js	JustAnotherPanel API client
server/telegram.js	Optional scrape of public channel posts
server/db.js	JSON file store (data/db.json)
public/*	Control panel (status, posts, curve, live)

3. Core goal (per post)

Metric	
Total paid views	~1,200
Duration	~4.5 hours (4–5h band)
Order size	Always 20 (JAP service 7394)
Instant open	~76 !’ 4×20 every 2s at start
Multi-post	Each link runs its own schedule

Approx milestones: 5 min !’ ~240 · 20 min !’ ~440 · 1h !’ ~660 · 2.5h !’ ~900 · 4.5h !’ ~1,200

4. Curve engine (curve.js) — full logic

Views are not one big order. They are many small +20 orders timed like real readership: fast at open, slower later.

4.1 Phases

Phase			
instant	8s	76 !’ 4 orders	Every 2s (notify pop)
notify	~5m	160	Max gap 30s
hot	15m	200	Max gap 70s
open	40m	220	Wider gaps
mid	90m	240	Up to ~5 min
finish	2h	300	Up to ~6 min

4.2 How a schedule is built

- For each phase: orders = ceil(phase.views / 20)
- intervalSec = duration / orders (early phases may force denser timing via maxGapSec / fillGaps)
- Instant is special: always ceil(76/20)=4 orders every 2 seconds
- Each drop: { at: startTime + age, qty: 20, phaseId, seq }

Result: dozens of timed +20 drops summing to ~1,200 over ~4.5h.

5. Lifecycle of one post

```
You paste link  !’  normalize URL  !’  buildSchedule(startAt)
    !’ each drop = pending | skipped (if >45s late)
    !’ save post in db.json  !’  status = active

Worker every 2 seconds:
  for each active post (round-robin fairness):
    take up to 4 due pending drops
    if LIVE !’ jap.addOrder(service, link, 20)
    if DRY  !’ fake order id DRY_...
    mark drop sent, log order

When no pending left !’ status = completed
```

- Late tracking: drops more than 45s past due become skipped; fresh adds still get the instant pack.
- Multi-post: independent curves; round-robin + max 4 drops/post/tick prevents starvation.
- Failures: live errors retry up to 3 times (~20s delay), then mark failed.

6. Dry-run vs Live

Mode	
DRY-RUN (default)	Full schedule + logs + fake IDs — no spend
LIVE	Real JAP add orders — uses balance

Toggle: UI switch !' POST /api/settings !' settings.liveMode (or LIVE_MODE in .env on boot).

7. How posts enter the system

- Manual — paste one or many https://t.me/... links in the dashboard.
- Sync channel — scrape t.me/s/{CHANNEL_USERNAME}; only posts from last 6 hours.
- Auto-discover — same scrape every 2 minutes if enabled.

If Telegram is blocked, auto-discover turns off; paste links manually — drip still works.

8. Data model (data/db.json)

```
posts[]:
  id, link, title, startAt, status (active|paused|completed)
  totalPlanned, totalOrdered
  drops[]: { at, qty:20, status, japOrderId, phaseId, seq, ... }

orders[]: history of each +20 submission
events[]: activity log
settings: liveMode, autoDiscover, channelUsername, serviceId, orderQty
```

9. Worker execution params

Parameter	
Tick interval	Every 2 seconds
Max drops per post / tick	4
Live gap between API calls	200ms
Miss window (late skip)	45 seconds
Retry delay / max attempts	20s / 3

10. External dependency: JustAnotherPanel

```
POST https://justanotherpanel.com/api/v2
  key + action=add + service=7394 + link + quantity=20

Also: action=balance | services | status
Credential: JAP_API_KEY in .env
```

11. End-to-end flow

- Start app !’ Express + worker timer + optional cron sync.
- Link in !’ schedule of many +20 timed drops.
- Worker fires each drop when now >= drop.at.
- LIVE !’ JAP buys 20 views on that post URL.
- UI polls every ~8s: progress, balance, event log, curve preview.

12. Formula summary

```
per post:
  instant:  4 × 20 every 2s          "H  80 views
  + notify / hot / open / mid / finish phases
  % % %
```

14. Setup

```
cd "d:\Robo Views Bot"
npm install
npm start

Open http://localhost:3847
Paste post links (one per line). Leave start time empty for now.
```

15. What this system is not

- Not a Telegram Bot API client for artificially counting views.
- Does not hack Telegram — only places SMM panel orders.
- Channel scrape is optional; core product is drip + JAP.

End of document — Robo Views Bot system analytic